Algorithm for generating a unique ID in C++?

Asked 12 years, 11 months ago Modified 5 years, 4 months ago Viewed 87k times

37

What can be the best algorithm to generate a unique id in C++? The length ID should be a 32 bit unsigned integer.

c++ uniqueidentifier

Share Improve this question Follow

edited Jan 1, 2010 at 15:53
Peter Mortensen
30.7k 21 104 125

asked Jan 1, 2010 at 14:05
Ajay
9,727 7 31 34

14 Unique in what sense - on your machine, in your application? – anon Jan 1, 2010 at 14:07

12 SLaks - how can you say that without knowing his requirements? And since when is a GUID 32 bits? – anon Jan 1, 2010 at 14:10

To be honest,what is the difference between the two of them? – Ajay Jan 1, 2010 at 14:15

Provide more information. The length of the ID a 32-bit integer - does this mean a 4-billion-digit ID? Unlikely. – Eli Bendersky Jan 1, 2010 at 14:16

4 @Ajay: as for the differene - if that's in your application then a simple counter will do the job. If that's on the machine then you want it unique even between different processes running the same program, and this is more difficult. – Eli Bendersky Jan 1, 2010 at 14:25

6 Answers

Sorted by: Highest score (default)

68

Getting a unique 32-bit ID is intuitively simple: the next one. Works 4 billion times. Unique for 136 years if you need one a second. The devil is in the detail: what was the previous one? You need a reliable way to persist the last used value and an atomic way to update it.

How hard that will be depends on the scope of the ID. If it is one thread in one process then you only need a file. If it is multiple threads in one process then you need a file and a mutex. If is multiple processes on one machine then you need a file and a named mutex. If it is multiple processes on multiple machines then you need to assign a authoritative ID provider, a single server that all machines talk to. A database engine is a common provider like that, they have this built-in as a feature, an auto-increment column.

The expense of getting the ID goes progressively up as the scope widens. When it becomes impractical, scope is Internet or provider too slow or unavailable then you need to give up on a 32-bit value. Switch to a random value. One that's random enough to make the likelihood

Join Stack Overflow to find the best answer to your technical question, help others answer theirs. Sign up

Share Improve this answer Follow edited Jun 22, 2012 at 7:29 answered Jan 1, 2010 at 16:29

Hans Passant
910k 145 1666 2496

Nice answer for the overview of the problem, any pointers to a standard method for the last solution, with the random number? What is a good practical approach to generate good enough random numbers? – overlii May 26, 2015 at 13:04

Maybe you were referring to "UUID", as described by this wikipedia article? en.wikipedia.org/wiki/... – overlii May 26, 2015 at 13:07

Yes. Whether a number is globally unique (guid) or universally unique (uuid) is something I don't want to touch with a ten-foot pole. It is a big universe out there :) – Hans Passant May 26, 2015 at 13:12

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up ×

14

Here's the simplest ID I can think of.

```
MyObject obj;
uint32_t id = reinterpret_cast<uint32_t>(&obj);
```

At any given time, this ID will be unique across the application. No other object will be located at the same address. Of course, if you restart the application, the object may be assigned a new ID. And once the object's lifetime ends, another object may be assigned the same ID.

And objects in different memory spaces (say, on different computers) may be assigned identical IDs.

And last but not least, if the pointer size is larger than 32 bits, the mapping will not be unique.

But since we know nothing about what kind of ID you want, and how unique it should be, this seems as good an answer as any.

Share Improve this answer Follow

edited Jan 1, 2010 at 20:12
Peter Mortensen
30.7k 21 104 125

answered Jan 1, 2010 at 18:01
jalf
240k 51 339 548

1 Bad idea. Imagine a 64 bits application. There is a very large chance that 32 bits of that address will be 0x00000000, or at least the same for all object addresses in a program. (Note: truncating reinterpret_cast results are likely to depend on endianness) – MSalters Jan 4, 2010 at 15:31

8 I already pointed that out. If the pointer size is larger than 32 bits, you'll get collisions. But since we know virtually nothing about what the ID is needed for, or in which type of application, I just wanted to suggest the simple and obvious solution. – jalf Jan 4, 2010 at 16:50

1 Is there a way for this to accommodate this to work for 64 bits and 32 bits? – dev_nut Mar 14, 2016 at 14:55

6 Be careful with this method. I used the same approach and when you remove your object and create new, new object can has the same address. It is too dangerous if you have multi-threading application. – Unick Apr 18, 2016 at 13:49

8

You can see this. (Complete answer, I think, is on Stack Overflow.)
Some note for unique id in C++ in Linux in this site. And you can use uuid in Linux, see this man page and sample for this.

If you use windows and need windows APIs, see this MSDN page.

This Wikipedia page is also useful: http://en.wikipedia.org/wiki/Universally_Unique_Identifier.

Share Improve this answer Follow

edited May 23, 2017 at 12:17
Community Bot
1 1

answered Jan 1, 2010 at 14:29
Sajad Bahmani
17.3k 27 85 108

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up ×

1

```
DWORD uid = ::GetTickCount();
::Sleep(100);
```

Share Improve this answer Follow

edited Feb 9, 2012 at 16:54
Lightness Races in Orbit
374k 74 635 1034

answered Jan 1, 2010 at 14:51
an0nym0usc0ward
1,177 7 8

1 Sorry, in theory, if two machines do this on the same time - both machine will get the same uid, so the number is not unique across computers. See Neil's comment to the main question. It's correct if the demand is only to get uniqueness on the same machine across time, and not uniqueness across machines. – elcuco Jan 1, 2010 at 15:49

6 Plus GetTickCount() is not very precise. You could call it three times in a row and get the same response. – John Dibling Jan 1, 2010 at 17:07

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up

0

There is little context, but if you are looking for a unique ID for objects within your application you can always use a singleton approach similar to

```
class IDGenerator {
   public:
      static IDGenerator * instance ();
      uint32_t next () { return _id++; }
   private:
      IDGenerator () : _id(0) {}

      static IDGenerator * only_copy;
      uint32_t _id;
}

IDGenerator *
IDGenerator::instance () {
   if (!only_copy) {
      only_copy = new IDGenerator();
   }
   return only_copy;
}
```

And now you can get a unique ID at any time by doing:

```
IDGenerator::instance()->next ()
```

Share Improve this answer Follow

answered Jan 1, 2010 at 16:30

ezpz
11.6k 6 37 39

3 static IDGenerator &instance () { static IDGenerator Generator; return Generator; } – ROAR Jan 1, 2010 at 22:23

Original answer allows for destruction of the singleton. Local static var is faster to implement but you lose that option. – Cross_ May 24, 2013 at 18:15

0

If you can afford to use Boost, then there is a UUID library that should do the trick. It's very straightforward to use - check the documentation and this answer.

Share Improve this answer Follow

answered Aug 16, 2017 at 12:31

Sahib Yar
976 11 27

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up